

CAHIER DES CHARGES

Projet N°7 — Système HIPS sous Linux

Conception, développement et déploiement d'un système de détection et de prévention d'intrusions hôte (HIPS) sous Linux avec interface web, intégrant analyse des événements, simulation d'attaques et mécanismes de réponse automatique.

Équipe — Groupe 2

- KAMDOUM TEUFENG Josette
- HARVIS LANDRY FOTSEU FONKWA
- Nonga Bikim

Encadrant :

NGUIMBUS Emmanuel

Année académique

2025-2026



Structure du cahier des charges



Introduction

Présentation générale du projet et du document



Contexte

Environnement Linux, menaces et limites des solutions actuelles



Définition du problème

Problématique centrale et sous-problèmes identifiés



Objectifs visés

Objectif général et 8 objectifs spécifiques mesurables



Description des fonctionnalités

Les 5 modules du système HIPS détaillés



Résultat attendu

Livrables documentaires et critères d'acceptation



Démarche d'intervention

Les 6 phases du projet avec calendrier



Ressources utilisées

Humaines, matérielles, logicielles et budgets prévisionnels



Suivi et validation

Modalités de suivi et critères de validation finale

Introduction

Présentation générale du projet et du document

Pourquoi ce projet est important

Le contexte de la cybersécurité aujourd'hui

Dans un monde où la transformation numérique s'accélère, la sécurité des systèmes d'information est devenue une préoccupation stratégique pour toute organisation. Les cyberattaques ne cessent de croître en nombre, en sophistication et en impact. Selon les rapports récents de cybersécurité, les intrusions sur des systèmes Linux représentent une part croissante des incidents signalés, notamment dans les environnements serveurs.

Les systèmes Linux, malgré leur robustesse reconnue, sont exposés à des vecteurs d'attaque variés : tentatives de connexion par force brute via SSH, accès non autorisés à des fichiers critiques du système, exécution de logiciels malveillants ou encore élévations de privilèges. Ces menaces sont d'autant plus dangereuses qu'elles peuvent passer inaperçues pendant des jours, voire des semaines, si aucun mécanisme de surveillance active n'est en place.

C'est dans ce contexte que s'inscrit notre projet : concevoir un système HIPS (Host Intrusion Prevention System) opérationnel sous Linux, capable de surveiller, détecter et contrer automatiquement ces menaces, avec une interface web de supervision accessible en temps réel.

Ce que ce cahier des charges définit

Ce document formalise l'ensemble des besoins et contraintes du projet. Il constitue le premier livrable officiel de la phase d'Étude et servira de référence tout au long du cycle de développement. Il a pour objectifs de :

- 1 Définir avec précision le contexte dans lequel le projet s'inscrit
- 2 Identifier et formaliser la problématique centrale ainsi que les sous-problèmes associés
- 3 Lister les objectifs à atteindre, assortis d'indicateurs mesurables
- 4 Décrire en détail les fonctionnalités attendues du système à réaliser
- 5 Préciser les résultats et livrables attendus à l'issue du projet
- 6 Présenter la démarche méthodologique adoptée pour la conduite du projet
- 7 Inventorier les ressources nécessaires à la réalisation, accompagnées d'un budget prévisionnel
- 8 Définir les modalités de suivi et les critères d'acceptation du projet

Section I

Contexte

Environnement Linux, menaces actuelles et limites des solutions existantes

Omniprésence et criticité des systèmes Linux

Linux est aujourd'hui le système d'exploitation le plus utilisé dans les environnements serveurs à travers le monde. Des études récentes estiment que plus de 90% des serveurs cloud et des infrastructures critiques fonctionnent sous Linux. Cette adoption massive s'explique par plusieurs facteurs fondamentaux :

Stabilité et fiabilité

Linux est reconnu pour sa capacité à fonctionner sans redémarrage pendant des mois, voire des années, ce qui en fait le choix privilégié pour les serveurs de production.

Modèle open source

La transparence du code source permet un audit continu par la communauté mondiale, favorisant la détection et la correction rapide des vulnérabilités.

Performances et légèreté

Son noyau optimisé consomme peu de ressources système, permettant de maximiser les performances des applications hébergées.

Sécurité native

Le modèle de permissions Unix, les namespaces, les cgroups et les capabilities offrent un niveau de contrôle granulaire difficile à égaler.

Types d'attaques recensées et leur impact

La popularité de Linux en fait paradoxalement une cible de choix pour les attaquants. Voici les principales catégories de menaces auxquelles les systèmes Linux sont confrontés :

Attaques par force brute (Brute Force)

Ces attaques consistent à tenter de deviner un mot de passe ou une clé d'authentification en essayant systématiquement toutes les combinaisons possibles. Sur Linux, le service SSH (port 22) est la cible principale. Un attaquant peut tenter des milliers de combinaisons par minute à l'aide d'outils spécialisés comme Hydra ou Medusa. Ces attaques génèrent des milliers de lignes de logs d'authentification et peuvent aboutir à une prise de contrôle complète du serveur si le mot de passe est faible.

Accès non autorisés à des fichiers sensibles

Certains fichiers Linux contiennent des informations critiques : `/etc/shadow` (hachages des mots de passe), `/etc/passwd` (comptes utilisateurs), les clés privées SSH (`/root/.ssh/`), les fichiers de configuration des services. Un utilisateur mal intentionné ou un processus compromis peut tenter d'y accéder pour extraire des informations d'identification ou modifier la configuration du système à des fins malveillantes.

Exécution de malwares et processus suspects

Les malwares ciblant Linux incluent des rootkits (qui se cachent dans le noyau), des mineurs de cryptomonnaie (qui épuisent les ressources CPU), des reverse shells (qui ouvrent une connexion vers un serveur de contrôle) et des ransomwares. Ces programmes sont souvent déposés dans des répertoires temporaires (`/tmp`) ou exécutés avec des permissions élevées obtenues via une élévation de privilèges.

Section II

Définition du problème

Problématique centrale et sous-problèmes détaillés

La question fondamentale à laquelle répond ce projet

« Comment détecter en temps réel les intrusions et comportements malveillants sur un système Linux hôte, et y répondre automatiquement avant qu'ils ne causent des dommages irréversibles sur l'infrastructure ? »

Cette problématique soulève quatre grands défis techniques et organisationnels auxquels ce projet doit répondre de manière concrète et opérationnelle :

1

Collecte et centralisation

Comment agréger efficacement les données de surveillance provenant de sources multiples (syslog, auditd) en temps réel sans perturber les performances du système hôte ?

2

Analyse et détection

Comment distinguer automatiquement un comportement normal d'un comportement malveillant dans un flux massif d'événements système ?

3

Réponse automatisée

Comment déclencher des contre-mesures proportionnelles et efficaces de manière entièrement automatique dans un délai suffisamment court pour prévenir tout dommage ?

4

Validation et test

Comment s'assurer de la fiabilité du système HIPS en l'absence d'attaques réelles, tout en minimisant les risques de faux positifs qui pourraient perturber le fonctionnement normal ?

Section III

Objectifs visés

Objectif général et objectifs spécifiques mesurables

Ce que le projet doit accomplir globalement

Concevoir, développer et déployer un système HIPS sous Linux, capable de collecter et analyser les journaux système en temps réel, de détecter les comportements malveillants, d'alerter via une interface web et de déclencher des contre-mesures automatiques.

Pour atteindre cet objectif, le système devra satisfaire simultanément aux exigences suivantes :

Surveillance continue	Fonctionner 24h/24, 7j/7 sans interruption de service ni dégradation notable des performances du système hôte
Détection multi-vecteurs	Couvrir au minimum 3 types distincts de menaces : force brute, accès fichiers sensibles et exécution de processus suspects
Réponse automatique	Déclencher des contre-mesures sans intervention humaine dans un délai inférieur à 30 secondes après détection
Traçabilité complète	Conserver un historique horodaté de tous les événements, alertes et actions effectuées par le système
Accessibilité	Offrir une interface web intuitive permettant à un administrateur de superviser l'état du système depuis n'importe quel navigateur

Objectifs 1 à 4 — Surveillance et détection

1 Mettre en place la collecte automatique des journaux système

Le système doit collecter de manière continue et automatique les journaux produits par deux sources : syslog (événements généraux, authentications, démarrages de services) et auditd (appels système, accès aux fichiers, modifications de droits). Les logs collectés doivent être normalisés dans un format commun, horodatés précisément et stockés dans une base de données locale pour permettre leur analyse et leur consultation ultérieure.

2 Détecter les attaques par force brute SSH

Le moteur de détection doit identifier automatiquement les tentatives de connexion SSH répétées depuis la même adresse IP sur une courte période. La règle de détection doit être configurable mais la valeur par défaut est fixée à 5 échecs d'authentification consécutifs en moins de 60 secondes. Dès que ce seuil est atteint, une alerte de niveau CRITIQUE doit être générée et la contre-mesure de blocage IP doit être déclenchée automatiquement.

3 Détecter les accès non autorisés aux fichiers sensibles

Le système doit surveiller en permanence les tentatives de lecture ou d'écriture sur des fichiers critiques du système et tout fichier de configuration critique des services. Toute tentative d'accès par un utilisateur non autorisé ou par un processus non référencé doit générer une alerte immédiate.

4 Détecter l'exécution de processus suspects

Le moteur doit détecter le lancement de processus inhabituels, notamment : les binaires exécutés depuis des répertoires temporaires (/tmp, /dev/shm), les scripts shell lancés avec des privilèges élevés, les processus consommant anormalement le CPU (indicateur possible de minage), et les processus ouvrant des connexions réseau sortantes non autorisées. Une liste blanche de processus légitimes doit être maintenue pour réduire les faux positifs.

Objectifs 5 à 8 — Interface, réponse et validation

5 Développer un dashboard web de supervision en temps réel

L'interface web doit offrir une vision synthétique et actualisée de l'état de sécurité du système. Elle doit afficher les alertes dès leur génération, avec toutes les informations nécessaires (type d'attaque, adresse IP source, horodatage, niveau de sévérité, action prise). Un module d'historique doit permettre de consulter et filtrer tous les événements passés. L'interface doit être sécurisée par authentification.

6 Implémenter le blocage automatique des adresses IP malveillantes

Lorsqu'une attaque de type force brute ou accès non autorisé est détectée depuis une adresse IP spécifique, le système doit automatiquement ajouter une règle de blocage dans iptables (ou nftables). Ce blocage doit être effectif immédiatement, sans intervention humaine. L'adresse IP bloquée et la raison du blocage doivent être enregistrées en base de données et affichées sur le dashboard. Un mécanisme de déblocage manuel doit être accessible via l'interface.

7 Implémenter l'arrêt automatique des processus malveillants

Lorsqu'un processus est identifié comme malveillant par le moteur de détection, le système doit envoyer automatiquement un signal SIGKILL au processus concerné pour l'arrêter immédiatement. L'identifiant du processus (PID), son nom, l'utilisateur qui l'a lancé et la raison de l'arrêt doivent être consignés dans les logs du HIPS et visibles sur le dashboard.

8 Réaliser et valider un module de simulation d'attaques

Le projet doit inclure un module permettant de simuler les trois types d'attaques définis (brute force SSH, accès à /etc/shadow, exécution de malware simple). Ces simulations sont obligatoires pour valider le bon fonctionnement de l'ensemble de la chaîne détection-réponse. Les résultats de chaque simulation (détecté ou non, délai de détection, contre-mesure déclenchée) doivent être documentés dans le rapport de déploiement.

Section IV

Description des fonctionnalités

Les 5 modules du système HIPS détaillés

Collecte et normalisation des journaux système

Description du module

Ce module constitue le point d'entrée du système HIPS. Il est responsable de la surveillance continue des deux sources de journaux principales du système Linux et de leur intégration dans la base de données du HIPS.

syslog

Le journal système général contient des informations sur les événements du système d'exploitation : démarrages et arrêts de services, messages du noyau, activités d'authentification (via PAM), erreurs des applications et notifications des démons système. Le module surveille ce fichier en temps réel grâce à un mécanisme de lecture de type tail -f implémenté en Python, détectant chaque nouvelle entrée dès son écriture.

auditd

Le démon d'audit Linux (auditd) fournit un niveau de traçabilité bien plus fin que syslog. Il permet de journaliser les appels système (syscalls), les accès aux fichiers (lecture, écriture, exécution), les changements de droits (chmod, chown), les connexions réseau et les modifications des règles d'audit. Le module configure auditd avec des règles adaptées aux menaces ciblées (surveillance de /etc/shadow, /etc/passwd, détection d'exécution depuis /tmp).

Normalisation et stockage

Les entrées des deux sources sont parsées, normalisées dans un format JSON structuré (timestamp, source, type_event, utilisateur, processus, fichier_cible, ip_source) et stockées dans la base de données SQLite/MySQL. Ce stockage permet une consultation historique et une analyse par le moteur de détection. Un système de rotation des logs évite une croissance incontrôlée de la base de données.

Analyse comportementale et identification des menaces

Description du module

Le moteur de détection est le composant central et le plus critique du système HIPS. Il analyse en continu le flux d'événements collectés et applique des règles de corrélation pour identifier les comportements malveillants. Voici le détail de chaque règle de détection :

Règle 1 — Détection force brute SSH

Le moteur compte les échecs d'authentification SSH provenant de la même adresse IP dans une fenêtre temporelle glissante de 60 secondes. Seuil : 5 échecs. Dès que ce seuil est atteint, une alerte de niveau CRITIQUE est générée, incluant l'IP source, le nombre de tentatives, la première et dernière tentative. La règle est réinitialisée après déclenchement pour ne pas générer de faux positifs en boucle.

Règle 2 — Détection accès fichiers sensibles

Une liste de fichiers et répertoires sensibles est maintenue en configuration : `/etc/shadow`, `/etc/passwd`, `/etc/sudoers`, `/root/.ssh/`, les certificats SSL, les fichiers de configuration des services (MySQL, Apache, Nginx). Toute opération de lecture (`OPEN_READ`) ou d'écriture (`OPEN_WRITE`) sur ces fichiers par un utilisateur ou processus non figurant dans la liste blanche déclenche une alerte de niveau ÉLEVÉ.

Règle 3 — Détection processus et malwares suspects

Le moteur surveille les événements d'exécution de processus (`EXECVE` dans `auditd`). Un processus est qualifié de suspect si : il est lancé depuis `/tmp`, `/dev/shm` ou `/var/tmp` ; son nom ne figure pas dans la liste blanche des processus légitimes ; il est exécuté avec des droits root par un utilisateur non administrateur ; il consomme plus de 80% CPU pendant plus de 30 secondes sans justification connue.

Interface de supervision en temps réel

Description du module

Le dashboard web est l'interface principale de l'administrateur système. Développé avec un framework web Python (Flask ou Django), il offre une vision centralisée de l'état de sécurité du système et permet de gérer les alertes et les contre-mesures.

Page d'accueil (vue synthétique)

Affiche les indicateurs clés en temps réel : nombre total d'alertes du jour, nombre d'IP bloquées, état du service HIPS (actif/inactif), charge CPU et mémoire du serveur, derniers événements. Un système de code couleur (vert/orange/rouge) permet d'évaluer instantanément le niveau de menace.

Module Alertes (temps réel)

Liste les alertes actives avec pour chaque alerte : le niveau de sévérité (INFO, WARNING, CRITICAL), le type d'attaque détectée, l'adresse IP source (si applicable), le nom du processus concerné, le fichier ciblé, l'horodatage et le statut de la contre-mesure déclenchée. Les alertes sont actualisées automatiquement sans rechargement de page (WebSocket ou polling AJAX).

Module Historique

Permet de consulter l'ensemble des événements passés avec des filtres par date, type d'événement, niveau de sévérité et statut. Offre la possibilité d'exporter l'historique au format CSV pour archivage ou audit. Affiche également la liste des IP bloquées avec la possibilité de les débloquent manuellement.

Authentification et sécurité

L'accès au dashboard est protégé par un formulaire de login avec identifiant et mot de passe hashé (bcrypt). Les sessions sont gérées avec des tokens sécurisés. La communication peut être sécurisée par HTTPS avec un certificat auto-signé. Un journal des connexions au dashboard est maintenu pour audit.

Module 4 — Réponse automatique aux menaces

Blocage IP (iptables)

Dès qu'une attaque de type force brute est détectée, le module exécute automatiquement une commande iptable, une règle bloque toute communication avec l'IP malveillante, dans les deux sens, de manière immédiate et sans intervention humaine. La règle et son horodatage sont enregistrés en base.

Kill process (SIGKILL)

Lorsqu'un processus malveillant est identifié, le module récupère son PID depuis les logs auditd et exécute `kill -9 <PID>`. Cette action arrête immédiatement le processus sans lui laisser la possibilité de libérer ses ressources ou d'exécuter des actions de nettoyage. Le PID, le nom du processus, l'utilisateur et la raison de l'arrêt sont tous consignés.

Notification et journalisation

Chaque action automatique déclenche la création d'une alerte de type ACTION dans la base de données, immédiatement visible sur le dashboard. Un fichier de log dédié aux actions du HIPS (`/var/log/hips/actions.log`) est maintenu pour audit. Ces logs d'actions sont eux-mêmes protégés en écriture pour empêcher leur falsification par un attaquant.

Module 5 — Simulation d'attaques (obligatoire)

Brute force SSH

Script automatisé utilisant Hydra ou un script Python custom effectuant N tentatives de connexion SSH par seconde sur le serveur cible, depuis la machine attaquante. Paramètres configurables : IP cible, nombre de tentatives, intervalle. Valide la règle de détection et le blocage IP automatique.

Accès non autorisé

Commande lancée depuis un compte utilisateur standard tentant de lire `/etc/shadow` via cat ou d'écrire dans `/root/.ssh/`. Valide la surveillance auditd des fichiers sensibles et la génération d'alerte correspondante.

Exécution malware simple

Script shell déposé dans `/tmp` et exécuté, simulant un comportement de reverse shell basique (connexion sortante vers une IP de contrôle) ou de consommation anormale de CPU. Valide la détection de processus suspect et le kill automatique.

Section V

Résultat attendu

Livrables documentaires et critères d'acceptation du projet

Les 6 documents à produire tout au long du projet

Conformément au référentiel pédagogique, chaque phase du projet doit se conclure par la remise d'un livrable documentaire. Ces livrables constituent la traçabilité officielle de l'avancement du projet et seront soumis à validation par l'encadrant.

1	Phase 1 — Étude	Cahier des charges	Document de définition du projet : contexte, problématique, objectifs, fonctionnalités, démarche, ressources et critères d'acceptation. C'est le présent document.
2	Phase 2 — Analyse	Dossier d'analyse	Modélisation complète des besoins : diagrammes de cas d'utilisation UML, diagrammes de séquence, diagrammes d'activité, dictionnaire des données et contraintes non fonctionnelles.
3	Phase 3 — Conception	Dossier de conception	Architecture logicielle détaillée (composants, interfaces, flux de données), schéma de la base de données, maquettes fonctionnelles de l'interface web, choix techniques justifiés.
4	Phase 4 — Réalisation	Dossier de réalisation	Documentation technique du code source, résultats des tests unitaires, captures d'écran des interfaces développées, guide d'installation et de configuration du système.
5	Phase 5 — Déploiement	Rapport de déploiement	Procédures d'installation sur la machine cible, résultats des tests d'intégration, rapport des simulations d'attaques (délais de détection, contre-mesures déclenchées, taux de détection).
6	Phase 6 — Clôture	Rapport final du projet	Bilan complet du projet : objectifs atteints/non atteints, difficultés rencontrées, solutions apportées, retours d'expérience, perspectives d'amélioration et recommandations.

Section VI

Démarche d'intervention

Les 6 phases du projet avec calendrier détaillé

Étape	Durée prévue	Date de début	Date de fin
Rédaction du cahier des charges	2 semaines	04/05/2026	17/05/2026
Analyses	3 semaines	18/05/2026	06/06/2026
Conception	2 semaine	07/06/2026	21/06/2026
Mise en oeuvre	2 semaine	22/06/2026	05/07/2026

Section VII

Ressources utilisées

Humaines, matérielles, logicielles + budgets prévisionnels détaillés

Estimation du coût de l'investissement humain

Poste	Nombre de personnes	Taux hebdomadaire (FCFA)	Durée (semaines)	Coût total (FCFA)
Chef de projet	1	250 000	9	2 250 000
Ingénieur Cybersécurité	1	300 000	9	2 700 000
Développeur Backend	1	250 000	6	1 500 000
Développeur Frontend	1	200 000	4	800 000
Administrateur Système	1	250 000	4	1 000 000
Pentester	1	300 000	2	600 000
Total	6	—	—	8 850 000 FCFA

Infrastructure nécessaire à la réalisation et au déploiement

La réalisation du projet nécessite un ensemble d'équipements matériels permettant de disposer d'un environnement de développement, de test et de déploiement représentatif d'un contexte professionnel réel.

Serveur Linux (machine hôte HIPS)

Ubuntu Server 20.04 LTS ou Debian 11. Configuration recommandée : processeur 2 cœurs minimum, 4 Go de RAM (8 Go recommandé), 50 Go de disque dur. C'est sur cette machine que sera déployé et testé l'agent HIPS, le moteur de détection et le dashboard web. Elle devra être accessible en réseau local depuis la machine attaquante et les postes de développement.

Machine attaquante (simulation d'attaques)

Poste sous Kali Linux ou machine virtuelle (VirtualBox/VMware) depuis laquelle seront lancées les simulations d'attaques : brute force SSH avec Hydra, tentative d'accès aux fichiers sensibles, exécution du malware simulé. Cette machine doit être dans le même réseau local que le serveur HIPS pour que les attaques soient détectables.

Infrastructure réseau locale

Un switch réseau 8 ports (ou un routeur WiFi) permettant de connecter les différentes machines en réseau local. Des câbles RJ45 de catégorie 5e minimum. Cette infrastructure est nécessaire pour simuler des attaques réseau réalistes (connexions SSH, accès web au dashboard). Dans un environnement de test, des machines virtuelles en réseau NAT ou bridge peuvent remplacer ce matériel.

Postes de développement

Les 3 membres de l'équipe disposent chacun d'un ordinateur personnel avec un système d'exploitation quelconque (Windows, Linux ou macOS), un navigateur web récent pour accéder au dashboard, un client SSH pour se connecter au serveur, Git installé pour le versionnage, et un éditeur de code (VS Code recommandé). Ces postes sont déjà disponibles et ne génèrent pas de coût supplémentaire.

Estimation des coûts des équipements

Ce tableau présente l'estimation budgétaire des ressources matérielles nécessaires au projet.

Équipement	Spécifications techniques	Qté	Prix unit. (FCFA)	Total (FCFA)
Serveur Linux	Ubuntu 20.04 LTS, 32 Go RAM, 1 T SSD, CPU 8 cœurs	1	750 000	750 000
Machine attaquante	VM Kali Linux	1	0	0
Switch réseau 8 ports	Fast Ethernet 100Mbps minimum, RJ45	1	15 000	15 000
Câbles RJ45 Cat5e	Longueur 3m, lot de 5 câbles	1	4 000	4 000
Alimentation ondulée (UPS)	Protection contre les coupures pour le serveur	1	25 000	25 000
Postes de développement	Ordinateurs personnels des 3 membres	5	400 000	2 000 000

TOTAL RESSOURCES MATÉRIELLES

2 794 000 FCFA

Les ressources logicielles utilisées sont :

- **Système d'exploitation** : Ubuntu Server
- **Langages et frameworks** : Python, Flask (ou Django)
- **Collecte et analyse des logs** : Syslog, Auditd
- **Base de données** : PostgreSQL (ou MySQL)
- **Interface Web** : HTML, CSS, JavaScript
- **Outils de sécurité et de test** : Nmap, Hydra, Metasploit, Wireshark
- **Outils de gestion de version** : Git, GitHub
- **Outils de virtualisation et de conteneurisation** : VirtualBox, Docker
- **Outils de supervision et visualisation** : Grafana

Ressource logicielle	Quantité	Coût unitaire (FCFA)	Coût total (FCFA)	Ressource logicielle
Ubuntu Server	1	0	0	Ubuntu Server
Python, Flask/Django	1	0	0	Python, Flask/Django
Syslog, Auditd	1	0	0	Syslog, Auditd
PostgreSQL / MySQL	1	0	0	PostgreSQL / MySQL
Nmap, Hydra, Metasploit, Wireshark	1	0	0	Nmap, Hydra, Metasploit, Wireshark
Grafana	1	0	0	Grafana
	7	--	--	

TOTAL RESSOURCES LOGICIELLES**0 FCFA — Stack 100% open source**

Synthèse de l'ensemble des coûts prévisionnels

Ce tableau consolidé présente la synthèse complète du budget prévisionnel du projet, en regroupant toutes les catégories de ressources.

Catégorie de ressources	Montant (FCFA)
Ressources humaines	8 850 000
Ressources matérielles	2 794 000
Ressources logicielles	0
TOTAL GÉNÉRAL DU PROJET	11 644 000 FCFA

Suivi et validation

Modalités de suivi et critères d'acceptation de la validation finale

Comment le projet sera piloté et contrôlé

Un bon suivi de projet est essentiel pour garantir le respect du calendrier, la qualité des livrables et la cohésion de l'équipe. Les modalités suivantes seront appliquées tout au long du projet.

Réunions hebdomadaires d'avancement

Chaque semaine, l'équipe se réunit (en présentiel ou à distance) pour faire le point sur l'avancement de chaque membre, identifier les blocages et prendre les décisions nécessaires. Un compte-rendu de réunion est rédigé par le chef de projet et partagé avec l'encadrant. Ces comptes-rendus constituent une preuve du travail collectif et permettent de détecter les retards en amont.

Dépôt Git partagé et revues de code

L'ensemble du code source est versionné sur un dépôt Git (GitHub ou GitLab). Chaque fonctionnalité est développée sur une branche dédiée (feature/nom-fonctionnalité). Avant toute fusion dans la branche principale, une revue de code par au moins un autre membre de l'équipe est obligatoire. Cela garantit la qualité du code, facilite la détection des bugs et favorise le partage des connaissances.

Remise et validation des livrables par phase

À la fin de chaque phase, le livrable correspondant est soumis à l'encadrant pour validation. Aucune phase suivante ne peut commencer avant que le livrable de la phase précédente n'ait été validé (ou que des corrections aient été intégrées). Cette contrainte garantit que le projet avance sur des bases solides et documentées.

Journal de bord du projet

Un journal de bord est tenu à jour quotidiennement par chaque membre. Il enregistre les tâches accomplies, le temps passé, les difficultés rencontrées et les solutions trouvées. Ce journal est utilisé lors des réunions hebdomadaires et contribue à la rédaction des livrables finaux, notamment le rapport de clôture.